

Week	Objective	Key Tasks	Milestone & Verifiable Deliverable
Week 1	Codebase Consolidation & Routing	1. Initialize a unified React, Vite, and TypeScript project. 2. Install react-router-dom and Lucide icon packages. 3. Import component folders and styles from all 6 independent modules. 4. Establish role-based layouts (Admin, Volunteer, Parent, Hub) with consistent sidebars and navigation headers. 5. Configure standard public and private application routes.	A fully navigable frontend React Single Page Application (SPA) running locally. Clicking sidebar and menu links seamlessly routes between the Homepage, Signup flows, and Mock Dashboards without full-page reloads.
Week 2	Backend Architecture & Database Design	1. Set up a Node.js Express server in TypeScript. 2. Initialize and configure Prisma ORM with the PostgreSQL driver. 3. Design and model the relational database tables for Users, Students, Parents, Mentors, Hubs, Classes, Sessions, Availabilities, VolunteerHours, and Absences. 4. Apply migrations to the PostgreSQL database. 5. Create a robust database seed script containing realistic mock data.	A functional backend server responding to health check requests, connected to a fully migrated PostgreSQL database. Database tables are verified as active and populated with the seed script via Prisma Studio.
Week 3	Azure AD Authentication Integration	1. Create the App Registration inside the Microsoft Entra ID portal. 2. Implement the MSAL React provider on the frontend to allow user login. 3. Establish a React Auth Context to store and distribute active sessions and user roles. 4. Build a custom backend token validation middleware using jwks-rsa and jsonwebtoken. 5. Apply role-based API endpoint guards on the Express backend.	Successful login via Microsoft account on the React frontend. Protected API endpoints return 401 Unauthorized for requests without a valid token and accept authorized requests, returning active user roles.
Week 4	Registration, Onboarding & User Profiles	1. Connect registration forms (Hub, Parent, and Volunteer signup flows) to the backend API. 2. Create secure POST endpoints for user profile storage. 3. Implement backend classification logic to automatically group student signups into age-appropriate brackets (Juniors: 8-10, Intermediates: 11-14, Seniors: 15-18). 4. Save structured user profiles and relationships in the PostgreSQL database.	A functioning end-to-end registration flow. Submitting a signup form from the public homepage saves user data to PostgreSQL and places students in the correct learning category based on age.
Week 5	Availability Management & Timezones	1. Integrate interactive weekly calendar grids on the frontend for volunteers, hubs, and students to select scheduling availability. 2. Establish backend endpoints to ingest and standardize availability data. 3. Build database utility functions to normalize schedules into UTC. 4. Develop timezone conversion helpers to support multiple localized timezone interfaces.	Users can save weekly availability preferences (e.g. EST, GMT) on their dashboard. The data is accurately converted to and stored in UTC, and successfully rendered back in the user's local timezone.
Week 6	Smart Matching & Automation Engine	1. Build the Smart Matching Core Engine backend algorithm to find availability overlaps between mentors and learners. 2. Implement pairing constraints: match 1 Primary Mentor, 1 Backup Mentor, and 3-5 students of the same age category and overlapping time window. 3. Build API endpoints to generate matching proposals. 4. Hook up the Admin Smart Matching UI to trigger the engine, view proposals, override pairings, and click Approve Match.	Clicking Run Matching Engine in the Admin portal instantly evaluates availability tables and displays pairing proposals. Approving a proposal successfully commits the permanent class structure to the database.
Week 7	Session Management & Teams Integration	1. Request and register delegated/application Microsoft Graph API permissions in the Azure portal. 2. Implement the Microsoft Graph SDK Client inside the Express backend. 3. Build automated event handlers to create a recurring weekly Microsoft Teams meeting when a class match is approved by an admin. 4. Save the generated Teams meeting links, IDs, and calendar invites to the database under Sessions. 5. Set up transactional emails to distribute join links to mentors and parents.	Approving a class match automatically generates a live, recurring Microsoft Teams meeting. The unique join link is successfully written to the PostgreSQL database, and a test email containing the link is successfully sent.

Week 8	Dashboards, Communication & Hours Tracking	1. Connect the Volunteer Dashboard and Parent/Hub Dashboard to pull active class schedules, rosters, and Teams links from the API. 2. Build an automated cron job on the backend to trigger hours-logging reminders to mentors after scheduled sessions. 3. Implement the volunteer hours logging form. 4. Build aggregation endpoints that calculate and output total impact metrics (sessions delivered, hours logged, participants reached).	Volunteers and parents see live class schedules and active Teams links on their dashboards. Volunteers can log hours post-session, and the Homepage and Admin impact stats dynamically update to reflect the new data.
Week 9	Absence & Replacement Workflows	1. Connect the Report Absence form on the Volunteer Dashboard to the API. 2. Write backend triggers for planned and emergency absences: automatically promote the backup mentor to primary for that session. 3. Develop the broadcast system to automatically email eligible replacement mentors matching the class age and timezone. 4. Implement a Claim Session feature in the volunteer portal that assigns the session on a first-come, first-served basis.	Submitting an absence request automatically promotes the backup mentor and sends an email alert to other eligible volunteers. A replacement mentor claiming the slot instantly updates the schedule in real-time.
Week 10	Safeguarding Audits, Testing & Production Launch	1. Build the Admin Safeguarding dashboard to manage parent consent tracking and log/review safety incident reports. 2. Conduct comprehensive end-to-end user acceptance testing (UAT) across all user roles (Student, Volunteer, Hub, Admin). 3. Host the PostgreSQL database on Azure Database for PostgreSQL (production). 4. Deploy the Express API to Azure App Service (production). 5. Deploy the React SPA to Azure Static Web Apps (production).	The platform is fully integrated, deployed, and live. Users can register, authenticate via Microsoft, be matched, participate in Teams meetings, log hours, and manage operations on the production domains.